

Bismillah General Store

Complete Beginner Developer Guide

A comprehensive documentation for understanding the entire codebase

Project: Online Grocery Store

Version: 1.0.0

Date: July 2026

Table of Contents

1. Project Overview

2. Technology Stack

3. Project Structure

4. Frontend Code Explanation

5. Backend Code Explanation

6. Database Models

7. API Routes

8. Authentication System

9. State Management (Context API)

10. How to Run the Project

11. Common Concepts for Beginners

12. Tips for New Developers

1. Project Overview

Bismillah General Store is a full-stack e-commerce web application for an online grocery store. It allows customers to browse products, add items to cart, place orders, and chat with support. Admins can manage products, orders, users, and view analytics.

Main Features

- **Customer Features:** Browse products, search/filter, add to cart, checkout, view orders, live chat support
- **Admin Features:** Dashboard with analytics, manage products/categories/orders/users, stock management, sales/bills records, chat support
- **Authentication:** Register, login, password recovery, role-based access (admin/customer)
- **Real-time Chat:** Customers can chat with admin support

Architecture: This project follows a **Client-Server Architecture** where the frontend (React) communicates with the backend (Express.js) through REST APIs. The backend connects to MongoDB database for data storage.

2. Technology Stack

Frontend Client Side

Technology	Version	Purpose
React	19.2.6	UI library for building user interfaces
Vite	8.0.12	Build tool and development server
React Router DOM	7.15.0	Client-side routing (navigation)
Ant Design (antd)	6.3.7	UI component library
Bootstrap	5.3.8	CSS framework for styling
Chart.js	4.5.1	Charts for dashboard analytics
Lottie React	2.4.1	Animations
SASS	1.99.0	CSS preprocessor

Backend Server Side

Technology	Version	Purpose
Node.js	-	JavaScript runtime
Express.js	4.21.2	Web framework for building APIs
Mongoose	8.9.5	MongoDB object modeling (ODM)
Socket.IO	4.8.1	Real-time bidirectional communication
JWT (jsonwebtoken)	9.0.2	Authentication tokens

Bcrypt.js	2.4.3	Password hashing
Multer	1.4.5	File upload handling
Cloudinary	2.5.1	Image storage and optimization
CORS	2.8.5	Cross-origin resource sharing

Database **Data Storage**

Technology	Purpose
MongoDB	NoSQL database for storing all application data
MongoDB Atlas	Cloud-hosted MongoDB service

3. Project Structure

```

D:\Muhammad Umar\
├─ index.html           # Main HTML entry point
├─ package.json         # Frontend dependencies & scripts
├─ vite.config.js       # Vite configuration
├─ eslint.config.js     # Code linting rules
├─
├─ src/                # Frontend source code
│   ├── main.jsx        # App entry point (renders React)
│   ├── App.jsx         # Main App component
│   ├── App.scss        # Global styles
│   ├──
│   ├── config/         # Configuration files
│   │   ├── api.js      # API client (fetch wrapper)
│   │   ├── firebase.js # Firebase placeholder
│   │   ├── global.js   # Global utility functions
│   │   └─ supabase.js  # Supabase placeholder
│   ├──
│   ├── context/        # React Context (State Management)
│   │   ├── AppProvider.jsx # Combines all context providers
│   │   ├── Auth.jsx      # Authentication state & methods
│   │   └─ Cart.jsx      # Shopping cart state & methods
│   ├──
│   ├── components/     # Reusable UI components
│   │   ├── Header/     # Navigation bar
│   │   ├── Footer/     # Footer with links
│   │   ├── Chat/       # Chat widget component
│   │   └─ Misc/        # Utility components (Loader, 404, ProtectedRoute)
│   ├──
│   ├── pages/          # Page components (routes)
│   │   ├── Routes.jsx  # Main routing configuration
│   │   ├── Auth/       # Login, Register, ForgotPassword
│   │   ├── Frontend/   # Public pages (Home, Shop, Cart, etc.)
│   │   └─ Dashboard/   # Admin panel pages
│   ├──
│   ├── utils/          # Utility/helper functions
│   │   ├── format.js   # Date & currency formatting
│   │   ├── chat.js     # Chat API functions
│   │   └─ recycleBin.js # Recycle bin operations
│   ├──
│   ├── scss/           # SASS stylesheets
│   └─ assets/          # Static assets (images, icons)

```

```

|
├─ server/                                # Backend source code
|   ├─ index.js                          # Server entry point
|   ├─ package.json                      # Backend dependencies
|   ├─ .env                             # Environment variables (secrets)
|   |
|   ├─ config/                           # Server configuration
|   |   ├─ db.js                         # MongoDB connection
|   |   └─ cloudinary.js                 # Cloudinary configuration
|   |
|   ├─ models/                           # Database schemas (Mongoose)
|   |   ├─ User.js                       # User model
|   |   ├─ Product.js                    # Product model
|   |   ├─ Category.js                   # Category model
|   |   ├─ Order.js                      # Order model
|   |   ├─ Chat.js                       # Chat model
|   |   └─ StockMovement.js              # Stock tracking model
|   |
|   ├─ routes/                           # API route handlers
|   |   ├─ auth.js                       # Authentication routes
|   |   ├─ products.js                   # Product CRUD routes
|   |   ├─ categories.js                 # Category routes
|   |   ├─ orders.js                     # Order routes
|   |   ├─ users.js                      # User management routes
|   |   ├─ dashboard.js                  # Dashboard analytics routes
|   |   ├─ stock.js                      # Stock management routes
|   |   ├─ chat.js                       # Chat routes
|   |   └─ upload.js                     # File upload routes
|   |
|   ├─ middleware/                       # Express middleware
|   |   ├─ auth.js                       # JWT authentication middleware
|   |   └─ upload.js                     # Multer file upload middleware
|   |
|   └─ uploads/                           # Temporary file uploads
|
├─ Data/                                # Documentation & data files
|   └─ Bismillah_General_Store_Complete_Documentation.pdf
|
└─ public/                              # Static public assets

```


4. Frontend Code Explanation

4.1 Entry Point: main.jsx

src/main.jsx

This is the **first file that runs** when the app starts. It:

- Imports React and creates the root element
- Wraps the app with `BrowserRouter` (for navigation)
- Wraps with `AppProvider` (for state management)
- Imports Bootstrap CSS and global configurations

```
// Simplified version
import App from './App.jsx'
import { BrowserRouter } from 'react-router-dom'
import AppProvider from './context/AppProvider.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <AppProvider>
        <App />
      </AppProvider>
    </BrowserRouter>
  </StrictMode>
)
```

For Beginners: `StrictMode` is a React feature that helps catch potential problems. `BrowserRouter` enables URL-based navigation without page reloads.

4.2 Main App Component: App.jsx

src/App.jsx

The root component that:

- Checks if the app is still loading (checking auth state)
- Shows a loading screen while checking authentication
- Renders the `Routes` component when ready
- Wraps everything with Ant Design's `ConfigProvider` for theming

4.3 Routing: `Routes.jsx`

`src/pages/Routes.jsx`

Defines **which page shows for each URL**:

URL Pattern	Component	Description
<code>/*</code>	Frontend	Public store pages (Home, Shop, etc.)
<code>/auth/*</code>	Auth	Login, Register, Forgot Password
<code>/dashboard/*</code>	Dashboard	Admin panel (protected)
<code>*</code>	NoPage	404 page not found

Important: The `ProtectedRoute` component checks if user is logged in and has admin role before allowing access to dashboard.

4.4 Frontend Pages: `Frontend/index.jsx`

`src/pages/Frontend/index.jsx`

Contains all **public-facing pages** with Header and Footer:

URL	Page	Description
-----	------	-------------

/	Home	Landing page with hero, categories, featured products
/shop	Shop	All products with search, filter, sort
/shop/:id	ProductDetails	Single product detail page
/cart	Cart	Shopping cart with items
/checkout	Checkout	Order placement form
/orders	Orders	Customer order history
/about	About	About the store
/contact	Contact	Contact information

4.5 Home Page Example

`src/pages/Frontend/Home/index.jsx`

Shows how a typical React page works:

```
// 1. Import dependencies
import { useState, useEffect } from 'react' // React hooks
import { api } from '@config/api' // API client
import { Card, Col, Row } from 'antd' // UI components

// 2. Define the component
const Home = () => {
  // 3. State variables
  const [products, setProducts] = useState([])
  const [loading, setLoading] = useState(true)

  // 4. Fetch data when component loads
  useEffect(() => {
    const fetchData = async () => {
      const data = await api.get('/products')
      setProducts(data)
      setLoading(false)
    }
    fetchData()
  })
}
```

```
}, []) // Empty array = run once on mount

// 5. Return JSX (what to display)
return (
  <div>
    {loading ? <Spinner /> : products.map(p => <Card key={p._id} />)}
  </div>
)
}
```

Key React Concepts:

- `useState()` - Creates a variable that React tracks for changes
- `useEffect(() => {}, [])` - Runs code when component first appears
- `async/await` - Handles asynchronous operations (like API calls)
- `JSX` - HTML-like syntax in JavaScript

4.6 Shop Page

`src/pages/Frontend/Shop/index.jsx`

Demonstrates **advanced state management**:

- Fetches all products from API
- Filters by category (from URL parameter)
- Search by product name
- Sort by price or newest
- Shows "Out of Stock" overlay for unavailable products

4.7 Cart Page

`src/pages/Frontend/Cart/index.jsx`

Uses the `useCart()` hook to:

- Display all items in cart

- Update quantities
- Remove items
- Calculate total price
- Show order summary

4.8 Navbar Component

```
src/components/Header/Navbar.jsx
```

The navigation bar that:

- Shows different buttons based on auth state (logged in/out)
- Shows cart badge with item count
- Admin sees "Dashboard" button, customer sees "My Orders"
- Mobile: Uses drawer for navigation
- Responsive design (desktop + mobile)

5. Backend Code Explanation

5.1 Server Entry Point: index.js

server/index.js

The backend server that:

- Creates an Express.js application
- Sets up Socket.IO for real-time chat
- Enables CORS (allows frontend to connect)
- Parses JSON and URL-encoded data
- Mounts all API routes
- Connects to MongoDB and starts listening

```
// Simplified server setup
import express from 'express'
import { createServer } from 'http'
import { Server } from 'socket.io'

const app = express() // Create Express app
const server = createServer(app) // Create HTTP server
const io = new Server(server) // Add Socket.IO

// Middleware
app.use(cors()) // Allow cross-origin requests
app.use(express.json()) // Parse JSON bodies

// Routes
app.use('/api/auth', authRoutes)
app.use('/api/products', productRoutes)
// ... more routes

// Connect to DB, then start server
connectDB().then(() => {
  server.listen(5000)
})
```

5.2 Database Connection: db.js

`server/config/db.js`

Connects to MongoDB using Mongoose. The connection string comes from `.env` file (environment variables).

Security: Never commit your `.env` file to Git! It contains secrets like database passwords and API keys.

5.3 Authentication Middleware: `auth.js`

`server/middleware/auth.js`

Protects routes by verifying JWT tokens:

- `protect` - Checks if user is logged in
- `adminOnly` - Checks if user is an admin
- `signToken()` - Creates a JWT token

```
// How middleware works
const protect = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1]
  if (!token) return res.status(401).json({ error: 'No token' })

  const decoded = jwt.verify(token, process.env.JWT_SECRET)
  req.user = decoded // Add user info to request
  next() // Continue to the actual route
}
```


6. Database Models (Mongoose Schemas)

Each model defines the **structure of data** stored in MongoDB.

6.1 User Model

```
server/models/User.js
```

Field	Type	Description
uid	String (unique)	Unique user identifier
fullName	String	User's full name
email	String (unique)	Email address
password	String	Hashed password (never plain text!)
phone	String	Phone number
avatar	String	Profile picture URL
role	String	'admin' or 'customer'
status	String	'active' or 'blocked'
isDeleted	Boolean	Soft delete flag

6.2 Product Model

```
server/models/Product.js
```

Field	Type	Description
name	String	Product name
description	String	Product description
price	Number	Selling price
costPrice	Number	Cost price (for admin)
stock	Number	Available quantity
unit	String	'KG', 'pcs', 'L', 'dozen', 'pack'
category	String	Category name
images	[String]	Array of image URLs
featured	Boolean	Show on homepage
position	Number	Sort order

6.3 Order Model

```
server/models/Order.js
```

Field	Type	Description
userId	String	Customer's user ID
items	[Mixed]	Array of products ordered

total	Number	Order total amount
status	String	'pending', 'confirmed', 'shipped', 'delivered', 'cancelled'
shipping	Object	Shipping address details

6.4 Category Model

```
server/models/Category.js
```

Field	Type	Description
name	String	Category name (e.g., "Fruits")
slug	String (unique)	URL-friendly version of name
icon	String	Emoji or icon
image	String	Category image URL

Soft Delete Pattern: Instead of actually deleting records, this project uses `isDeleted` flag. This allows recovery from recycle bin and maintains data integrity.

7. API Routes

7.1 Auth Routes

```
server/routes/auth.js
```

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Register new user	No
POST	/api/auth/login	Login user	No
GET	/api/auth/me	Get current user	Yes
PUT	/api/auth/change-password	Change password	Yes

7.2 Product Routes

```
server/routes/products.js
```

Method	Endpoint	Description	Auth Required
GET	/api/products	Get all products	No
GET	/api/products/:id	Get single product	No
POST	/api/products	Create product	Admin
PUT	/api/products/:id	Update product	Admin

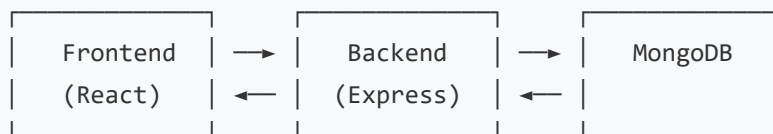
PUT	/api/products/:id	Soft delete product	Admin
PUT	/api/products/:id/restore	Restore product	Admin
DELETE	/api/products/:id/permanent	Permanent delete	Admin

7.3 Other Routes

Route File	Purpose
categories.js	CRUD for product categories
orders.js	Place orders, view orders, update status
users.js	Admin: list/block/unblock users
dashboard.js	Analytics data (total sales, orders, etc.)
stock.js	Stock movement tracking
chat.js	Chat messages CRUD
upload.js	File upload to Cloudinary

8. Authentication System

8.1 How Authentication Works



LOGIN FLOW:

1. User enters email + password
2. Frontend sends POST /api/auth/login
3. Backend checks credentials in database
4. If valid, backend creates JWT token
5. Token sent back to frontend
6. Frontend stores token in localStorage
7. All future requests include token in header

8.2 Frontend Auth Context

src/context/Auth.jsx

Manages authentication state across the entire app:

```
// State managed:
{
  isAuthenticated: false, // Is user logged in?
  user: null, // User object (name, email, role, etc.)
  userRole: null, // 'admin' or 'customer'
  isAppLoading: true // Still checking auth?
}

// Methods provided:
handleLogin(email, password) // Log in
handleRegister(name, email, pass, avatar) // Register
handleLogout() // Log out
```


8.3 Protected Routes

src/components/Misc/ProtectedRoute.jsx

```
const ProtectedRoute = ({ element, adminOnly }) => {  
  const { isAuthenticated, userRole, isLoading } = useAuth()  
  
  if (isLoading) return <ScreenLoader /> // Show loading  
  if (!isAuthenticated) return <Navigate to="/auth/login" /> // Redirect to login  
  if (adminOnly && userRole !== 'admin') return <Navigate to="/" /> // Not  
  admin  
  return element // All good, show the page  
}
```


9. State Management (Context API)

9.1 What is Context API?

Context API allows sharing data between components **without passing props manually** through every level. Think of it like a "global variable" that any component can access.

9.2 AppProvider Structure

src/context/AppProvider.jsx

```
// Wraps two context providers together
const AppProvider = ({ children }) => {
  return (
    <AuthContext> { /* Authentication state */ }
    <CartContext> { /* Shopping cart state */ }
    {children}
    </CartContext>
    </AuthContext>
  )
}
```

9.3 Cart Context

src/context/Cart.jsx

Manages shopping cart with **localStorage persistence**:

```
// Cart state is saved to localStorage
// So it persists even after page refresh!

const [cart, setCart] = useState(() => {
  const saved = localStorage.getItem('cart')
  return saved ? JSON.parse(saved) : []
})

// Methods available:
```

```
addToCart(product, quantity) // Add item
removeFromCart(productId) // Remove item
updateQuantity(productId, qty) // Change quantity
clearCart() // Empty cart
cartTotal // Calculate total price
cartCount // Total items count
```

9.4 Using Context in Components

```
// In any component:
import { useAuth } from '@context/Auth'
import { useCart } from '@context/Cart'

const MyComponent = () => {
  const { isAuth, user, handleLogout } = useAuth()
  const { cart, cartCount, addToCart } = useCart()

  return (
    <div>
      {isAuth ? `Hello ${user.fullName}` : 'Please login'}
      Cart has {cartCount} items
    </div>
  )
}
```


10. How to Run the Project

10.1 Prerequisites

- Node.js (v18 or higher) - [Download](#)
- npm (comes with Node.js)
- MongoDB (local or MongoDB Atlas account)
- Code editor (VS Code recommended)

10.2 Setup Steps

Step 1: Clone/Download the project

```
# If using Git:  
git clone <repository-url>  
cd Bismillah-Store
```

Step 2: Install Frontend Dependencies

```
npm install
```

Step 3: Install Backend Dependencies

```
cd server  
npm install  
cd ..
```

Step 4: Configure Environment Variables

Create server/.env file:

```
PORT=5000  
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/store  
JWT_SECRET=your-secret-key-here  
FRONTEND_URL=http://localhost:5173
```

```
# Cloudinary (for image uploads)
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret
```

Step 5: Start Backend Server

```
cd server
npm run dev
```

Step 6: Start Frontend (in new terminal)

```
npm run dev
```

Step 7: Open Browser

```
Frontend: http://localhost:5173
Backend API: http://localhost:5000/api
```

10.3 Available Scripts

Command	Where	What it does
npm run dev	Root	Start frontend dev server
npm run build	Root	Build frontend for production
npm run lint	Root	Check code for errors
npm run dev	server/	Start backend with auto-reload
npm start	server/	Start backend server

11. Common Concepts for Beginners

11.1 React Components

Components are **reusable pieces of UI**. Think of them as LEGO blocks that you combine to build a page.

```
// Functional Component (modern way)
const Greeting = ({ name }) => {
  return <h1>Hello, {name}!</h1>
}

// Using it: <Greeting name="Ahmad" />
```

11.2 Props

Props are like **function arguments** - they pass data from parent to child component.

```
// Parent passes props:
<Product name="Apple" price={100} />

// Child receives props:
const Product = ({ name, price }) => {
  return <div>{name}: Rs {price}</div>
}
```

11.3 State

State is **data that changes over time**. When state changes, React re-renders the component.

```
const [count, setCount] = useState(0)

// Read: count (current value)
// Update: setCount(1) or setCount(prev => prev + 1)
```

11.4 Hooks

Hooks are special functions that let you "hook into" React features:

Hook	Purpose
useState	Add state to component
useEffect	Run code after render (like API calls)
useContext	Use values from Context
useRef	Access DOM elements directly
useNavigate	Programmatic navigation
useParams	Get URL parameters
useSearchParams	Get URL query parameters

11.5 JSX

JSX is **HTML in JavaScript**. It looks like HTML but has some differences:

```
// HTML: class="btn" → JSX: className="btn"  
// HTML: for="name" → JSX: htmlFor="name"  
// HTML: onclick → JSX: onClick  
// HTML: <img /> → JSX: <img /> (self-closing required)  
// JavaScript: {expression} inside JSX
```

11.6 REST API

APIs follow REST conventions:

HTTP Method	CRUD Operation	Example
GET	Read	GET /api/products

POST	Create	POST /api/products
PUT	Update	PUT /api/products/123
DELETE	Delete	DELETE /api/products/123

11.7 Async/Await

```
// Instead of callbacks:
fetch('/api/products')
  .then(res => res.json())
  .then(data => console.log(data))

// Use async/await (cleaner):
const fetchData = async () => {
  const res = await fetch('/api/products')
  const data = await res.json()
  console.log(data)
}
```

11.8 Environment Variables

Secret values stored in .env files (never committed to Git):

```
# server/.env
MONGODB_URI=mongodb+srv://...
JWT_SECRET=my-secret-key
CLOUDINARY_API_KEY=12345
```


12. Tips for New Developers

12.1 Getting Started

- **Read the code one file at a time** - Start with main.jsx, then App.jsx, then Routes.jsx
- **Run the app first** - Understand what it does before reading code
- **Use browser DevTools** - Inspect elements, check console for errors
- **Read error messages** - They usually tell you exactly what's wrong

12.2 Learning Path

1. Learn basic HTML, CSS, JavaScript
2. Learn React fundamentals (components, props, state, hooks)
3. Understand this project's structure
4. Learn Express.js basics (routes, middleware)
5. Understand MongoDB/Mongoose basics
6. Practice by making small changes

12.3 Useful Resources

- [React Documentation](#) - Official React docs
- [Express.js Guide](#) - Backend framework docs
- [Ant Design Docs](#) - UI component library
- [Bootstrap Docs](#) - CSS framework
- [Mongoose Docs](#) - MongoDB ODM

12.4 Common Mistakes to Avoid

- **Don't** commit .env files or node_modules
- **Don't** store passwords in plain text (always hash)

- **Don't** forget to handle errors in async functions
- **Don't** mutate state directly (use setState)
- **Don't** skip dependency arrays in useEffect

12.5 Debugging Tips

- Use `console.log()` to see variable values
- Check browser console for frontend errors
- Check terminal for backend errors
- Use VS Code debugger for step-by-step debugging
- Test API endpoints with Postman or curl

Remember: Every expert was once a beginner. Take it step by step, practice regularly, and don't be afraid to make mistakes. That's how you learn!

*Documentation prepared for Bismillah General Store Project
Version 1.0 - July 2026*